

Lecture 7

Fundamentals of Numerical Analysis

C. Mou

Department of Mathematics and Statistics
Utah State University

Fall 2026

What we'll do:

- Gaussian elimination

Gaussian Elimination — Cartoon Version

Summary

- Gaussian Elimination is an orderly process for transforming an augmented matrix into an equivalent upper triangular form.
- The elimination operation at the k^{th} step is

$$\tilde{a}_{ij} = \tilde{a}_{ij} - (\tilde{a}_{ik}/\tilde{a}_{kk})\tilde{a}_{kj}, \quad i > k, \quad j \geq k$$

- Elimination requires three nested loops.
- The result of the elimination phase is represented by the image below.

$$\begin{bmatrix} x & x & x & x & x \\ x & x & x & x & x \\ x & x & x & x & x \\ x & x & x & x & x \end{bmatrix} \longrightarrow \begin{bmatrix} x & x & x & x & x \\ 0 & x' & x' & x' & x' \\ 0 & 0 & x'' & x'' & x'' \\ 0 & 0 & 0 & x''' & x''' \end{bmatrix}$$

Gaussian Elimination

Summary

- Transform a linear system into (upper) triangular form. i.e. transform lower triangular part to zero
- Transformation is done by taking linear combinations of rows
- Example: $a = \begin{bmatrix} a_1 \\ a_2 \end{bmatrix}$
- If $a_1 \neq 0$, then

$$\begin{bmatrix} 1 & 0 \\ -a_2/a_1 & 1 \end{bmatrix} \begin{bmatrix} a_1 \\ a_2 \end{bmatrix} = \begin{bmatrix} a_1 \\ 0 \end{bmatrix}$$

Gaussian Elimination Algorithm

Listing 1: Forward Elimination beta

```
1  given  $A, b$ 
2
3  for  $k = 1 \dots n - 1$ 
4      for  $i = k + 1 \dots n$ 
5          for  $j = k \dots n$ 
6               $a_{ij} = a_{ij} - (a_{ik}/a_{kk})a_{kj}$ 
7          end
8       $b_i = b_i - (a_{ik}/a_{kk})b_k$ 
9  end
10 end
```

- the multiplier can be moved outside the j -loop
- no reason to actually compute 0

Challenge: The loops over i and j may be exchanged—why would one be preferable?

Gaussian Elimination Algorithm

Listing 2: Forward Elimination

```
1  given  $A, b$ 
2
3  for  $k = 1 \dots n - 1$ 
4      for  $i = k + 1 \dots n$ 
5           $xmult = a_{ik}/a_{kk}$ 
6           $a_{ik} = xmult$ 
7          for  $j = k + 1 \dots n$ 
8               $a_{ij} = a_{ij} - (xmult)a_{kj}$ 
9          end
10          $b_i = b_i - (xmult)b_k$ 
11     end
12 end
```

Naive Gaussian Elimination Algorithm

- Forward Elimination
- + Backward substitution
- = Naive Gaussian Elimination

Forward Elimination Cost?

What is the cost in converting from A to U ?

Step	Add	Multiply	Divide
1	$(n-1)^2$	$(n-1)^2$	$n-1$
2	$(n-2)^2$	$(n-2)^2$	$n-2$
$\vdots$			
$n-1$	1	1	1

or

add	$\sum_{j=1}^{n-1} j^2$
multiply	$\sum_{j=1}^{n-1} j^2$
divide	$\sum_{j=1}^{n-1} j$

Forward Elimination Cost?

add	$\sum_{j=1}^{n-1} j^2$
multiply	$\sum_{j=1}^{n-1} j^2$
divide	$\sum_{j=1}^{n-1} j$

We know $\sum_{j=1}^p j = \frac{p(p+1)}{2}$ and $\sum_{j=1}^p j^2 = \frac{p(p+1)(2p+1)}{6}$, so

add-subtracts	$\frac{n(n-1)(2n-1)}{6}$
multiply-divides	$\frac{n(n-1)(2n-1)}{6} + \frac{n(n-1)}{2} = \frac{n(n^2-1)}{3}$

Forward Elimination Cost?

add-subtracts	$\frac{n(n-1)(2n-1)}{6}$
multiply-divides	$\frac{n(n-1)}{2}$
add-subtract for b	$\frac{n(n-1)}{2}$
multiply-divides for b	$\frac{n(n-1)}{2}$

Back Substitution Cost

As before

add-subtract	$\frac{n(n-1)}{2}$
multipply-divides	$\frac{n(n+1)}{2}$

Naive Gaussian Elimination Cost

Combining the cost of forward elimination and backward substitution gives

add-subtracts	$\frac{n(n-1)(2n-1)}{6} + \frac{n(n-1)}{2} + \frac{n(n-1)}{2}$ $= \frac{n(n-1)(2n+5)}{3}$
multiply-divides	$\frac{n(n^2-1)}{3} + \frac{n(n-1)}{2} + \frac{n(n+1)}{2}$ $= \frac{n(n^2+3n-1)}{3}$

So the total cost of add-subtract-multiply-divide is about

$$\frac{2}{3}n^3$$

⇒ double n results in a cost increase of a factor of 8

Elimination Matrices

- Another way to zero out entries in a column of A
- Annihilate entries below k^{th} element in a with matrix, M_k :

$$M_k a = \begin{bmatrix} 1 & \dots & 0 & 0 & \dots & 0 \\ \vdots & \ddots & \vdots & \vdots & \ddots & \vdots \\ 0 & \dots & 1 & 0 & \dots & 0 \\ 0 & \dots & -m_{k+1} & 1 & \dots & 0 \\ \vdots & \ddots & \vdots & \vdots & \ddots & \vdots \\ 0 & \dots & -m_n & 0 & \dots & 1 \end{bmatrix} \begin{bmatrix} a_1 \\ \vdots \\ a_k \\ a_{k+1} \\ \vdots \\ a_n \end{bmatrix} = \begin{bmatrix} a_1 \\ \vdots \\ a_k \\ 0 \\ \vdots \\ 0 \end{bmatrix}$$

where $m_i = a_i/a_k$, $i = k+1, \dots, n$.

- The divisor a_k is the “pivot” (and needs to be nonzero)

Elimination Matrices

- Matrix M_k is an “elementary elimination matrix”
 - ▶ Adds a multiple of row k to each subsequent row, with “multipliers” m_i
 - ▶ Result is zeros in the k^{th} column for rows $i > k$.
- M_k is unit lower triangular and nonsingular
- $M_k = I - m_k e_k^T$ where $m_k = [0, \dots, 0, m_{k+1}, \dots, m_n]^T$ and e_k is the k^{th} column of the identity matrix I .
- $M_k^{-1} = I + m_k e_k^T$, which means M_k^{-1} is also lower triangular, and we will denote $M_k^{-1} = L_k$.

Can you prove $M_k^{-1} = I + m_k e_k^T$?

Elimination Matrices

- Suppose M_j and M_k are elementary elimination matrices with $j > k$, then

$$\begin{aligned}M_k M_j &= I - m_k e_k^T - m_j e_j^T + m_k e_k^T m_j e_j^T \\&= I - m_k e_k^T - m_j e_j^T + m_k (e_k^T m_j) e_j^T \\&= I - m_k e_k^T - m_j e_j^T\end{aligned}$$

because the k^{th} entry of vector m_j is zero (since $j > k$)

- Thus $M_k M_j$ is essentially a union of their columns.
- Note this is also true for $M_k^{-1} M_j^{-1}$.

Example

Let $a = \begin{bmatrix} 2 \\ 4 \\ -2 \end{bmatrix}$.

$$M_1 a = \begin{bmatrix} 1 & 0 & 0 \\ -2 & 1 & 0 \\ 1 & 0 & 1 \end{bmatrix} \begin{bmatrix} 2 \\ 4 \\ -2 \end{bmatrix} = \begin{bmatrix} 2 \\ 0 \\ 0 \end{bmatrix}$$

and

$$M_2 a = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 1/2 & 1 \end{bmatrix} \begin{bmatrix} 2 \\ 4 \\ -2 \end{bmatrix} = \begin{bmatrix} 2 \\ 4 \\ 0 \end{bmatrix}$$

Example

So

$$L_1 = M_1^{-1} = \begin{bmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ -1 & 0 & 1 \end{bmatrix}, \quad L_2 = M_2^{-1} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & -1/2 & 1 \end{bmatrix}$$

which means

$$M_1 M_2 = \begin{bmatrix} 1 & 0 & 0 \\ -2 & 1 & 0 \\ 1 & 1/2 & 1 \end{bmatrix}, \quad L_1 L_2 = \begin{bmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ -1 & -1/2 & 1 \end{bmatrix}$$

Gaussian Elimination

- To reduce $Ax = b$ to upper triangular form, first construct M_1 with a_{11} as the pivot (eliminating the first column of A below the diagonal.)
- Then $M_1Ax = M_1b$ still has the same solution.
- Next construct M_2 with pivot a_{22} to eliminate the second column below the diagonal.
- Then $M_2M_1Ax = M_2M_1b$ still has the same solution
- $M_{n-1} \dots M_1Ax = M_{n-1} \dots M_1b$
- Let $M = M_nM_{n-1} \dots M_1$. Then $MAx = Mb$, with MA upper triangular.
- Do back substitution on $MAx = Mb$.

Another Way to Look at A

We've mentioned L and U today. Why?
Consider this

$$A = A$$

$$A = (M^{-1}M)A$$

$$A = (M_1^{-1}M_2^{-1} \dots M_n^{-1})(M_nM_{n-1} \dots M_1)A$$

$$A = (M_1^{-1}M_2^{-1} \dots M_n^{-1})((M_nM_{n-1} \dots M_1)A)$$

$$A = \qquad \qquad L \qquad \qquad U$$

But MA is upper triangular, and we've seen that $M_1^{-1} \dots M_n^{-1}$ is lower triangular. Thus, we have an algorithm that factors A into two matrices L and U .

Why is this “naive”?

Example

$$A = \begin{bmatrix} 0 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

Example

$$A = \begin{bmatrix} 1e-10 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

The Need for Pivoting

Solve:

$$A = \begin{bmatrix} 2 & 4 & -2 & -2 \\ 1 & 2 & 4 & -3 \\ -3 & -3 & 8 & -2 \\ -1 & 1 & 6 & -3 \end{bmatrix} \quad b = \begin{bmatrix} -4 \\ 5 \\ 7 \\ 7 \end{bmatrix}$$

Note that there is nothing "wrong" with this system. A is full rank. The solution exists and is unique.

Form the augmented system.

$$\left[\begin{array}{cccc|c} 2 & 4 & -2 & -2 & -4 \\ 1 & 2 & 4 & -3 & 5 \\ -3 & -3 & 8 & -2 & 7 \\ -1 & 1 & 6 & -3 & 7 \end{array} \right]$$

The Need for Pivoting

Subtract $1/2$ times the first row from the second row,
add $3/2$ times the first row to the third row,
add $1/2$ times the first row to the fourth row.
The result of these operations is:

$$\left[\begin{array}{cccc|c} 2 & 4 & -2 & -2 & -4 \\ 0 & 0 & 5 & -2 & 7 \\ 0 & 3 & 5 & -5 & 1 \\ 0 & 3 & 5 & -4 & 5 \end{array} \right]$$

The *next* stage of Gaussian elimination will not work because there is a zero in the *pivot* location, $\tilde{a}_{22}$.

The Need for Pivoting

Swap second and fourth rows of the augmented matrix.

$$\left[\begin{array}{cccc|c} 2 & 4 & -2 & -2 & -4 \\ 0 & 3 & 5 & -4 & 5 \\ 0 & 3 & 5 & -5 & 1 \\ 0 & 0 & 5 & -2 & 7 \end{array} \right]$$

Continue with elimination: subtract (1 times) row 2 from row 3.

$$\left[\begin{array}{cccc|c} 2 & 4 & -2 & -2 & -4 \\ 0 & 3 & 5 & -4 & 5 \\ 0 & 0 & 0 & -1 & -4 \\ 0 & 0 & 5 & -2 & 7 \end{array} \right]$$

The Need for Pivoting

Another zero has appear in the pivot position. Swap row 3 and row 4.

$$\left[\begin{array}{cccc|c} 2 & 4 & -2 & -2 & -4 \\ 0 & 3 & 5 & -4 & 5 \\ 0 & 0 & 5 & -2 & 7 \\ 0 & 0 & 0 & -1 & -4 \end{array} \right]$$

The augmented system is now ready for backward substitution.

another example

$$\begin{bmatrix} \varepsilon & 1 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$$

Example

With Naive GE,

$$\begin{bmatrix} \varepsilon & 1 \\ 0 & (1 - \frac{1}{\varepsilon}) \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 1 \\ 2 - \frac{1}{\varepsilon} \end{bmatrix}$$

Solving for x_1 and x_2 we get

$$x_2 = \frac{2 - 1/\varepsilon}{1 - 1/\varepsilon}$$

$$x_1 = \frac{1 - x_2}{\varepsilon}$$

For $\varepsilon \approx 10^{-20}$, $x_1 \approx 0$, $x_2 \approx 1$

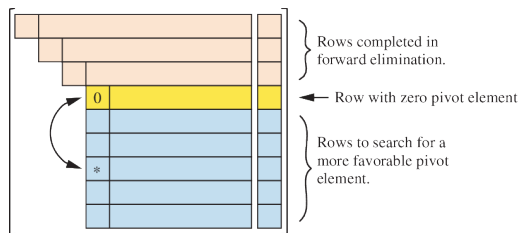
Pivoting Strategies

Partial Pivoting: Exchange only rows

- Exchanging rows does not affect the order of the x_i
- For increased numerical stability, make sure the largest possible pivot element is used. This requires searching in the partial column below the pivot element.
- Partial pivoting is usually sufficient.

Partial Pivoting

To avoid division by zero, swap the row having the zero pivot with one of the rows below it.



To minimize the effect of roundoff, always choose the row that puts the largest pivot element on the diagonal, i.e., find i_p such that $|a_{i_p,i}| = \max(|a_{k,i}|)$ for $k = i, \dots, n$

Partial Pivoting: Usually sufficient, but not always

- Partial pivoting is usually sufficient
- Consider

$$\left[\begin{array}{cc|c} 2 & 2c & 2c \\ 1 & 1 & 2 \end{array} \right]$$

With Partial Pivoting, the first row is the pivot row:

$$\left[\begin{array}{cc|c} 2 & 2c & 2c \\ 0 & 1-c & 2-c \end{array} \right]$$

and for large c :

$$\left[\begin{array}{cc|c} 2 & 2c & 2c \\ 0 & -c & -c \end{array} \right]$$

so that $y = 0$ and $x = 1$. (exact is $x = y = 1$)

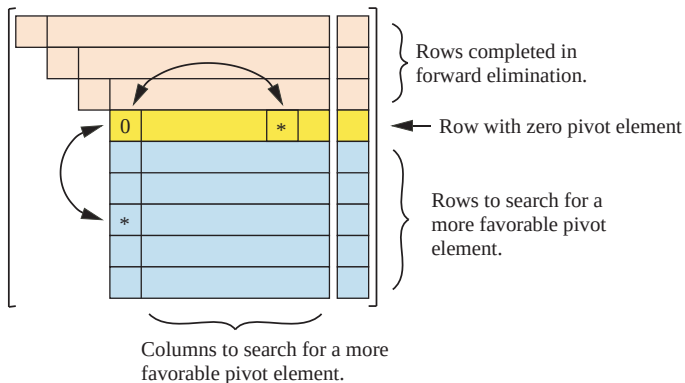
- The pivot is selected as the largest in the column, but it should be the largest relative to the full submatrix.

More Pivoting Strategies

Full (or Complete) Pivoting: Exchange *both* rows and columns

- Column exchange requires changing the order of the x_i
- For increased numerical stability, make sure the largest possible pivot element is used. This requires searching in the pivot row, *and* in all rows below the pivot row, starting the pivot column.
- Full pivoting is less susceptible to roundoff, but the increase in stability comes at a cost of more complex programming (not a problem if you use a library routine) and an increase in work associated with searching and data movement.

Full Pivoting



Scaled Partial Pivoting

We simulate full pivoting by using a scale with partial pivoting.

- pick pivot element as the largest **relative** entry in the column (relative to the other entries in the row)
- do not swap, just keep track of the order of the pivot rows
- call this vector $\ell = [\ell_1, \dots, \ell_n]$.

SPP Process

- 1 Determine a scale vector $\mathbf{s}$. For each row

$$s_i = \max_{1 \leq j \leq n} |a_{ij}|$$

- 2 initialize $\ell = [\ell_1, \dots, \ell_n] = [1, \dots, n]$.
- 3 select row j to be the row with the largest ratio

$$\frac{|a_{\ell_i 1}|}{s_{\ell_i}} \quad 1 \leq i \leq n$$

- 4 swap ℓ_j with ℓ_1 in ℓ
- 5 Now we need $n - 1$ multipliers for the first column:

$$m_1 = \frac{a_{\ell_i 1}}{a_{\ell_1 1}}$$

- 6 So the index to the rows are being swapped, NOT the actual row vectors which would be expensive
- 7 finally use the multiplier m_1 times row ℓ_1 to subtract from rows ℓ_i for $2 \leq i \leq n$

SPP Process continued

- 1 For the second column in forward elimination, we select row j that yields the largest ratio of

$$\frac{|a_{\ell_i,2}|}{s_{\ell_i}} \quad 2 \leq i \leq n$$

- 2 swap ℓ_j with ℓ_2 in ℓ
- 3 Now we need $n - 2$ multipliers for the second column:

$$m_2 = \frac{a_{\ell_i,2}}{a_{\ell_2,2}}$$

- 4 finally use the multiplier m_2 times row ℓ_2 to subtract from rows ℓ_i for $3 \leq i \leq n$
- 5 the process continues for row k
- 6 note: scale factors are *not* updated

An Example

Consider

$$\begin{bmatrix} 2 & 4 & -2 \\ 1 & 3 & 4 \\ 5 & 2 & 0 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 6 \\ -1 \\ 2 \end{bmatrix}$$

Back Substitution...

- 1 The first equation corresponds to the last index ℓ_n :

$$a_{\ell_n n} x_n = b_{\ell_n} \Rightarrow x_n = \frac{b_{\ell_n}}{a_{\ell_n n}}$$

- 2 The second equation corresponds to the second to last index ℓ_{n-1} :

$$x_{n-1} = \frac{1}{a_{\ell_{n-1} n-1}} (b_{\ell_{n-1}} - a_{\ell_{n-1} n} x_n)$$

The Algorithms

Listing 3: (forward) GE with SPP

```
1 Initialize  $\ell = [1, \dots, n]$ 
2 Set  $s$  to be the max of rows
3 for  $k = 1$  to  $n$ 
4    $rmax = 0$ 
5   for  $i = k$  to  $n$ 
6      $r = |a_{\ell_i k} / s_{\ell_i}|$ 
7     if ( $r > rmax$ )
8        $rmax = r$ 
9        $j = i$ 
10  end
11  swap  $\ell_j$  and  $\ell_k$ 
12  for  $i = k + 1$  to  $n$ 
13     $xmult = a_{\ell_i k} / a_{\ell_k k}$ 
14     $a_{\ell_i k} = xmult$ 
15    for  $j = k + 1$  to  $n$ 
16       $a_{\ell_i j} = a_{\ell_i j} - xmult \cdot a_{\ell_k j}$ 
17    end
18  end
19 end
```

The Algorithms

Note: the multipliers are stored in the location $a_{\ell_i k}$ in the text

Listing 4: (back solve) GE with SPP

```
1  for k = 1 to n - 1
2      for i = k + 1 to n
3           $b_{\ell_i} = b_{\ell_i} - a_{\ell_i k} b_{\ell_k}$ 
4      end
5  end
6   $x_n = b_{\ell_n} / a_{\ell_n n}$ 
7  for i = n - 1 down to 1
8       $sum = b_{\ell_i}$ 
9      for j = i + 1 to n
10          $sum = sum - a_{\ell_{ij}} x_j$ 
11     end
12 end
```

Geometric Interpretation of Singularity

Consider a 2×2 system describing two lines that intersect

$$y = -2x + 6$$

$$y = \frac{1}{2}x + 1$$

The matrix form of this equation is

$$\begin{bmatrix} 2 & 1 \\ -1/2 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 6 \\ 1 \end{bmatrix}$$

The equations for two **parallel** but **not intersecting** lines are

$$\begin{bmatrix} 2 & 1 \\ 2 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 6 \\ 5 \end{bmatrix}$$

Here the coefficient matrix is singular ($\text{rank}(A) = 1$), and the system is inconsistent

Geometric Interpretation of Singularity

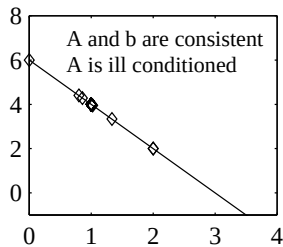
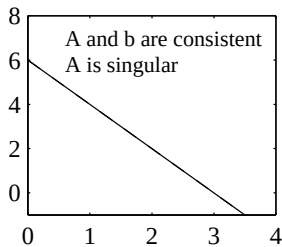
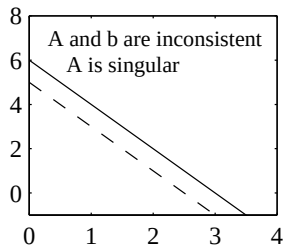
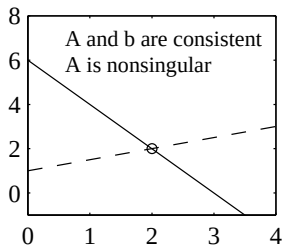
The equations for two **parallel** and **coincident** lines are

$$\begin{bmatrix} 2 & 1 \\ 2 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 6 \\ 6 \end{bmatrix}$$

The equations for two **nearly parallel** lines are

$$\begin{bmatrix} 2 & 1 \\ 2 + \delta & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 6 \\ 6 + \delta \end{bmatrix}$$

Geometric Interpretation of Singularity



Effect of Perturbations to b

Consider the solution of a 2×2 system where

$$b = \begin{bmatrix} 1 \\ 2/3 \end{bmatrix}$$

One expects that the *exact* solutions to

$$Ax = \begin{bmatrix} 1 \\ 2/3 \end{bmatrix} \quad \text{and} \quad Ax = \begin{bmatrix} 1 \\ 0.6667 \end{bmatrix}$$

will be different. Should these solutions be a **lot different** or a **little different**?

Norms

Vectors:

$$\|x\|_p = (|x_1|^p + |x_2|^p + \dots + |x_n|^p)^{1/p}$$

$$\|x\|_1 = |x_1| + |x_2| + \dots + |x_n| = \sum_{i=1}^n |x_i|$$

$$\|x\|_\infty = \max(|x_1|, |x_2|, \dots, |x_n|) = \max_i (|x_i|)$$

Matrices:

$$\|A\| = \max_{x \neq 0} \frac{\|Ax\|}{\|x\|}$$

$$\|A\|_p = \max_{x \neq 0} \frac{\|Ax\|_p}{\|x\|_p}$$

$$\|A\|_1 = \max_{1 \leq j \leq n} \sum_{i=1}^m |a_{ij}|$$

$$\|A\|_\infty = \max_{1 \leq i \leq m} \sum_{j=1}^n |a_{ij}|$$

Effect of Perturbations to b

Perturb b with δb such that

$$\frac{\|\delta b\|}{\|b\|} \ll 1,$$

The perturbed system is

$$A(x + \delta x_b) = b + \delta b$$

The perturbations satisfy

$$A\delta x_b = \delta b$$

Analysis shows (see next two slides for proof) that

$$\frac{\|\delta x_b\|}{\|x\|} \leq \|A\| \|A^{-1}\| \frac{\|\delta b\|}{\|b\|}$$

Thus, the effect of the perturbation is small *only if* $\|A\| \|A^{-1}\|$ is small.

$$\frac{\|\delta x_b\|}{\|x\|} \ll 1 \quad \text{only if} \quad \|A\| \|A^{-1}\| \sim 1$$

Effect of Perturbations to b (Proof)

Let $x + \delta x_b$ be the *exact* solution to the perturbed system

$$A(x + \delta x_b) = b + \delta b \quad (1)$$

Expand

$$Ax + A\delta x_b = b + \delta b$$

Subtract Ax from left side and b from right side since $Ax = b$

$$A\delta x_b = \delta b$$

Left multiply by A^{-1}

$$\delta x_b = A^{-1}\delta b \quad (2)$$

Effect of Perturbations to b (Proof, p. 2)

Take norm of equation (2)

$$\|\delta x_b\| = \|A^{-1} \delta b\|$$

Applying consistency requirement of matrix norms

$$\|\delta x\| \leq \|A^{-1}\| \|\delta b\| \quad (3)$$

Similarly, $Ax = b$ gives $\|b\| = \|Ax\|$, and

$$\|b\| \leq \|A\| \|x\| \quad (4)$$

Rearrangement of equation (4) yields

$$\frac{1}{\|x\|} \leq \frac{\|A\|}{\|b\|} \quad (5)$$

Effect of Perturbations to b (Proof)

Multiply Equation (4) by Equation (3) to get

$$\frac{\|\delta x_b\|}{\|x\|} \leq \|A\| \|A^{-1}\| \frac{\|\delta b\|}{\|b\|} \quad (6)$$

Summary:

If $x + \delta x_b$ is the *exact* solution to the perturbed system

$$A(x + \delta x_b) = b + \delta b$$

then

$$\frac{\|\delta x_b\|}{\|x\|} \leq \|A\| \|A^{-1}\| \frac{\|\delta b\|}{\|b\|}$$

Effect of Perturbations to A

Perturb A with δA such that

$$\frac{\|\delta A\|}{\|A\|} \ll 1,$$

The perturbed system is

$$(A + \delta A)(x + \delta x_A) = b$$

Analysis shows that

$$\frac{\|\delta x_A\|}{\|x + \delta x_A\|} \leq \|A\| \|A^{-1}\| \frac{\|\delta A\|}{\|A\|}$$

Thus, the effect of the perturbation is small *only if* $\|A\| \|A^{-1}\|$ is small.

$$\frac{\|\delta x_A\|}{\|x + \delta x_A\|} \ll 1 \quad \text{only if} \quad \|A\| \|A^{-1}\| \sim 1$$

Effect of Perturbations to both A and b

Perturb both A with δA and b with δb such that

$$\frac{\|\delta A\|}{\|A\|} \ll 1 \quad \text{and} \quad \frac{\|\delta b\|}{\|b\|} \ll 1$$

The perturbation satisfies

$$(A + \delta A)(x + \delta x) = b + \delta b$$

Analysis shows that

$$\frac{\|\delta x\|}{\|x + \delta x\|} \leq \frac{\|A\| \|A^{-1}\|}{1 - \|A\| \|A^{-1}\| \frac{\|\delta A\|}{\|A\|}} \left[\frac{\|\delta A\|}{\|A\|} + \frac{\|\delta b\|}{\|b\|} \right]$$

Thus, the effect of the perturbation is small *only if* $\|A\| \|A^{-1}\|$ is small.

$$\frac{\|\delta x\|}{\|x + \delta x\|} \ll 1 \quad \text{only if} \quad \|A\| \|A^{-1}\| \sim 1$$

Condition number of A

The **condition number**

$$\kappa(A) \equiv \|A\| \|A^{-1}\|$$

indicates the sensitivity of the solution to perturbations in A and b . The condition number can be measured with any p -norm.

The condition number is always in the range

$$1 \leq \kappa(A) \leq \infty$$

- $\kappa(A)$ is a mathematical property of A
- Any algorithm will produce a solution that is sensitive to perturbations in A and b if $\kappa(A)$ is large.
- In exact math a matrix is either singular or non-singular.
 $\kappa(A) = \infty$ for a singular matrix
- $\kappa(A)$ indicates how close A is to being numerically singular.
- A matrix with large κ is said to be **ill-conditioned**

Computational Stability

In Practice, applying Gaussian elimination with partial pivoting and back substitution to $Ax = b$ gives the **exact solution**, $\hat{x}$, to the **nearby problem**

$$(A + E)\hat{x} = b \quad \text{where} \quad \|E\|_{\infty} \leq \varepsilon_m \|A\|_{\infty}$$

Gaussian elimination with partial pivoting and back substitution “gives exactly the right answer to nearly the right question.”

— Trefethen and Bau

Computational Stability

An algorithm that gives the exact answer to a problem that is near to the original problem is said to be **backward stable**. Algorithms that are not backward stable will tend to amplify roundoff errors present in the original data. As a result, the solution produced by an algorithm that is not backward stable will not necessarily be the solution to a problem that is close to the original problem.

Gaussian elimination without partial pivoting is *not* backward stable for arbitrary A .

If A is symmetric and positive definite, then Gaussian elimination without pivoting is backward stable.

The Residual

Let $\hat{x}$ be the numerical solution to $Ax = b$. $\hat{x} \neq x$ (x is the exact solution) because of roundoff.

The **residual** measures how close $\hat{x}$ is to satisfying the original equation

$$r = b - A\hat{x}$$

It is not hard to show that

$$\frac{\|\hat{x} - x\|}{\|\hat{x}\|} \leq \kappa(A) \frac{\|r\|}{\|b\|}$$

Small $\|r\|$ does not guarantee a small $\|\hat{x} - x\|$.

If $\kappa(A)$ is large the $\hat{x}$ returned by Gaussian elimination and back substitution (or any other solution method) is *not* guaranteed to be anywhere near the true solution to $Ax = b$.

Rules of Thumb

- Applying Gaussian elimination with partial pivoting and back substitution to $Ax = b$ yields a numerical solution $\hat{x}$ such that the residual vector $r = b - A\hat{x}$ is small *even if* the $\kappa(A)$ is large.
- If A and b are stored to machine precision ε_m , the numerical solution to $Ax = b$ by any variant of Gaussian elimination is correct to d digits where

$$d = |\log_{10}(\varepsilon_m)| - \log_{10}(\kappa(A))$$

Rules of Thumb

$$d = |\log_{10}(\varepsilon_m)| - \log_{10}(\kappa(A))$$

Example:

MATLAB computations have $\varepsilon_m \approx 2.2 \times 10^{-16}$. For a system with $\kappa(A) \sim 10^{10}$ the elements of the solution vector will have

$$\begin{aligned} d &= |\log_{10}(2.2 \times 10^{-16})| - \log_{10}(10^{10}) \\ &= 16 - 11 \\ &= 5 \end{aligned}$$

correct (decimal) digits

Summary of Limits to Numerical Solution of $Ax = b$

- 1 $\kappa(A)$ indicates how close A is to being numerically singular
- 2 If $\kappa(A)$ is “large”, A is **ill-conditioned** and *even the best* numerical algorithms will produce a solution, $\hat{x}$ that cannot be guaranteed to be close to the true solution, x
- 3 In practice, Gaussian elimination with partial pivoting and back substitution produces a solution with a small residual

$$r = b - A\hat{x}$$

even if $\kappa(A)$ is large.

The Backslash Operator

Consider the scalar equation

$$5x = 20 \quad \implies \quad x = (5)^{-1}20$$

The extension to a system of equations is, of course

$$Ax = b \quad \implies \quad x = A^{-1}b$$

where $A^{-1}b$ is the formal solution to $Ax = b$

In MATLAB notation the system is solved with

```
x = A\b
```

The Backslash Operator

Given an $n \times n$ matrix A , and an $n \times 1$ vector b the $\backslash$ operator performs a sequence of tests on the A matrix. MATLAB attempts to solve the system with the method that gives the least roundoff and the fewest operations.

When A is an $n \times n$ matrix:

- 1 MATLAB examines A to see if it is a permutation of a triangular system
If so, the appropriate triangular solve is used.
- 2 MATLAB examines A to see if it *appears* to be symmetric and positive definite.
If so, MATLAB attempts a Cholesky factorization and two triangular solves.
- 3 If the Cholesky factorization fails, or if A does not appear to be symmetric,
MATLAB attempts an LU factorization and two triangular solves.